Universidad Nacional de Tucumán

Facultad de Ciencias Exactas y Tecnología



Simulación de Probabilidad Colectiva (Monty Hall)

Por: Coronel, Francisco Exequiel (Ingeniería en Informática), Cosentino Veglia, Luciano (Licenciatura en Informática);

Repositorio: https://github.com/LucianoCV01/mnII_proyecto_2_monty_hall

2026

Métodos Numéricos II

Resumen

La Paradoja de Monty Hall afirma que, en un concurso de tres puertas, cambiar la elección inicial gana el doble de veces que mantenerla (2/3 contra 1/3). En este trabajo se comprobó ese resultado con una simulación de Monte Carlo escrita en C y paralelizada con MPI, para 3, 5, 10 y 50 puertas, y se midió su rendimiento en un clúster dado por la Facultad (hasta 28 procesos), en una Mac con procesador M1 (hasta 8 procesos) y en una PC con Windows (hasta 8 procesos). Con mil millones de partidas, la frecuencia obtenida coincidió con la teoría en todos los casos (0,666666 frente a 0,666667 con 3 puertas). Durante el desarrollo se detectó que la función rand() de macOS producía una diferencia constante de unas cuatro cienmilésimas con la teoría, y se la reemplazó por el generador SplitMix64. En el clúster, el programa fue unas 28 veces más rápido con 28 procesos para problemas grandes, y con problemas chicos la ganancia se frena antes (cerca de 15 procesos con diez mil partidas). En la Mac, el speedup fue casi ideal hasta 4 procesos y no mejoró con más, por la mezcla de núcleos rápidos y lentos. Se concluye que el problema se paraleliza muy bien y que la cantidad de procesos conveniente depende del tamaño del problema y del hardware.

Resumen **2**
1. Introducción **4**
1.1 Fundamento teórico 4
1.2 Objetivos 5
2. Datos y Experimentos **5**
2.1 Implementación 6
2.2 Cómo medimos el rendimiento 7
2.3 Cómo validamos los resultados 7
3. Resultados **7**
4. Conclusiones **15**
4.1 Limitaciones 16
4.2 Mejoras recomendadas 17
5. Referencias **17**

1. Introducción

La Paradoja de Monty Hall es un problema de probabilidad cuya solución resulta contraintuitiva. En el escenario clásico, un concursante enfrenta tres puertas cerradas: detrás de una hay un premio de gran valor y detrás de las otras dos, cabras. El jugador elige una puerta al azar. A continuación, el presentador, que conoce la ubicación del premio, abre una de las puertas no elegidas y revela siempre una opción perdedora. En ese momento se plantea el dilema central: ¿le conviene al jugador mantener su elección inicial o cambiar a la otra puerta que permanece cerrada? La intuición sugiere que, al quedar dos puertas, ambas son equiprobables (50% cada una), pero la teoría muestra que no es así.

Como el resultado desafía el sentido común, es un buen caso de estudio para validarlo mediante simulación masiva de eventos. Este trabajo desarrolla en C, con la librería de paso de mensajes MPI, un simulador paralelo de la paradoja. El simulador resuelve el caso clásico de 3 puertas y su generalización a M puertas (M = 3, 5, 10 y 50). Además de comparar sus resultados con la teoría, se evalúa cómo se comporta el programa al paralelizarse en tres sistemas de cómputo distintos.

1.1 Fundamento teórico

Probabilidad de cada estrategia

Sea M la cantidad de puertas. Si el jugador elige inicialmente la puerta con el premio (probabilidad 1/M), gana si mantiene su elección y pierde si cambia. Si elige una puerta sin premio (probabilidad (M − 1)/M), el presentador abre todas las demás puertas incorrectas excepto una, y la única puerta alternativa que queda cerrada es la que tiene el premio. En ese caso, cambiar gana siempre. Por lo tanto:

P(ganar manteniendo) = 1/M y P(ganar cambiando) = (M − 1)/M.

Para M = 3 resultan 1/3 y 2/3, es decir, cambiar duplica las posibilidades de éxito. Para M = 5, 10 y 50, cambiar gana con probabilidad 0,800; 0,900 y 0,980. Estos valores suponen que el presentador conoce el premio y abre todas las puertas vacías menos una.

Este razonamiento implica que cambiar gana exactamente cuando la elección inicial fue errónea. Esa equivalencia permite simular el juego sin reproducir al presentador, y es la que usa el programa: lo que se verifica empíricamente es la frecuencia con que la elección inicial coincide o no con el premio.

Simulación de Monte Carlo

El método de Monte Carlo consiste en repetir el juego muchas veces, contar las victorias de cada estrategia y comparar la frecuencia obtenida con la probabilidad teórica. Es el método que se usa en este trabajo para validar la paradoja.

Paralelismo

Como cada partida simulada es independiente de las demás, el problema se clasifica como *embarazosamente paralelo*: las partidas pueden repartirse entre varios procesos que no se comunican durante el cálculo, y solo hace falta sumar los resultados al final.

1.2 Objetivos

Objetivo general

Verificar, mediante una simulación paralela masiva (método de Monte Carlo), la frecuencia con la que gana cada estrategia en la Paradoja de Monty Hall, compararla con la teoría y evaluar el rendimiento del programa en tres sistemas de cómputo.

Objetivos específicos

1. **Implementar en C con MPI:** un simulador paralelo y parametrizable en la cantidad de partidas (N), de procesos (P) y de puertas (M), para el caso clásico de 3 puertas y para 5, 10 y 50 puertas.
2. **Validar los resultados contra la teoría:** comparar la frecuencia obtenida con $(M-1)/M$ y $1/M$, y comprobar que el error disminuye como $O(1/\sqrt{N})$ al aumentar N, sin desviaciones sistemáticas.
3. **Evaluar el rendimiento:** medir el speedup y la eficiencia (escalado fuerte) con la mediana de varias ejecuciones, variando N, P y M.
4. **Comparar tres arquitecturas:** contrastar la escalabilidad en una Mac M1 (núcleos de alto rendimiento y de bajo consumo) con la de un nodo de un clúster y PC con Windows, e interpretar por qué el programa escala de forma distinta en cada una.
5. **Documentar los problemas encontrados y proponer mejoras:** registrar los problemas surgidos durante el desarrollo y las mejoras para futuros trabajos.

2. Datos y Experimentos

El simulador no usa datos externos: genera las partidas en la memoria de cada proceso mientras se ejecuta, sin leer ni escribir archivos durante el cálculo. Cada ejecución produce una única línea de resultados (formato CSV) con el tiempo, las victorias de cada estrategia, la probabilidad obtenida y la semilla usada. No se guarda cada partida por separado porque, con miles de millones, la escritura a disco distorsionaría la medición del tiempo.

En los experimentos se variaron tres parámetros: la cantidad de partidas (N, de 10^4 a 10^9), la cantidad de procesos (P, de 1 a 8 en la Mac y de 1 a 28 en el clúster) y la cantidad de puertas (M = 3, 5, 10 y 50).

Tabla 1. Sistemas utilizados.

	Mac M1	**Clúster**	**PC-Windows**
Procesador	Apple M1: 4 núcleos de alto rendimiento y 4 de bajo consumo	AMD EPYC 7B12: núcleos de igual velocidad	AMD Ryzen 5 2600: 6 núcleos físicos, 12 hilos, núcleos iguales

Sistema operativo	macOS	Linux (Ubuntu 24.04)	Windows
Tipo de equipo	Computadora personal	Un solo nodo compartido: los procesos se comunican por memoria compartida, sin red	Computadora personal
Procesos usados	Hasta 8	Hasta 28	Hasta 8

Se hicieron los siguientes experimentos:

- **Experimento principal (Mac):** N de 10^5 a 10^9, P de 1 a 8, las cuatro cantidades de puertas y 10 repeticiones por combinación.
- **Experimento espejo (clúster):** los mismos parámetros, con 5 repeticiones, para comparar ambos sistemas.
- **Barrido continuo (clúster):** P de 1 a 28 sin saltear valores, N de 10^4 a 10^9, solo con 3 puertas y 3 repeticiones, para ver hasta dónde escala el programa.
- **Controles:** una versión serial (sin MPI) como línea base y una variante sin la barrera previa al cronómetro.

2.1 Implementación

El simulador está escrito en C con MPI. Cada proceso ejecuta el mismo programa sobre su parte de las partidas y no se comunica con los demás hasta el final. Las decisiones de diseño son las siguientes.

Reparto de la carga: Las N partidas se reparten en partes iguales entre los P procesos. Cuando N no es múltiplo de P, el resto se reparte de a una partida entre los primeros procesos. Con una división entera simple se perderían en silencio hasta $P - 1$ partidas.

Simulación de cada partida: Cada proceso sortea la puerta con el premio y la puerta elegida. Como el presentador siempre abre una puerta vacía que el jugador no eligió, cambiar gana exactamente cuando la elección inicial fue errónea, y mantener gana cuando coincide con el premio. Por eso el programa no reproduce al presentador: usa esta equivalencia, que se justificó en la sección 1.1. Lo que se verifica empíricamente es la frecuencia con la que la elección inicial acierta.

Memoria*:* Todo se genera al vuelo en la memoria local de cada proceso. Lo único que se guarda son dos contadores de victorias, de 64 bits, para que N pueda superar los 2.100 millones sin desbordar.

Suma de resultados: Al terminar, cada proceso suma sus victorias y se juntan en el proceso 0 con la operación colectiva MPI_Reduce. Se prefirió a enviar los mensajes uno por uno porque es más simple, evita errores de sincronización y deja a MPI la elección de la forma más eficiente de juntar los resultados.

Números aleatorios y semillas: La primera versión usaba rand() de la biblioteca estándar de C y obtenía la semilla con time(NULL). Ambas decisiones se corrigieron durante el desarrollo:

- **Generador.** rand() producía una diferencia constante con la teoría que solo se hace visible con cientos de millones de partidas (sección 3.2). Se lo reemplazó por una implementación propia de SplitMix64 (Steele, Lea y Flood, 2014), un generador simple, rápido y sin llamadas a la biblioteca.
- **Semilla.** time(NULL) tiene resolución de un segundo, por lo que corridas lanzadas en el mismo segundo repetían la semilla. Ahora el proceso 0 genera una semilla de 64 bits a partir del reloj en nanosegundos, la reparte a los demás con MPI_Bcast y la guarda en cada línea del CSV, de modo que cualquier corrida puede repetirse.
- **Independencia entre procesos.** Para que los procesos no reciban secuencias parecidas, cada uno deriva su estado inicial de la semilla base y de su número de proceso mediante SplitMix64.

2.2 Cómo medimos el rendimiento

El tiempo se mide con MPI_Wtime en el proceso 0, desde que arranca la parte paralela hasta que termina la suma final de las victorias. Como el proceso 0 espera a todos los demás, ese tiempo incluye al más lento. No incluye el arranque de los procesos. Antes de iniciar el cronómetro hay una única barrera (MPI_Barrier), para que todos arranquen juntos y el proceso 0 no empiece a medir antes o después que el resto. No hay barrera posterior, porque la suma final ya obliga al proceso 0 a esperar a todos. Para comprobar si la barrera previa era necesaria, se repitió parte de los experimentos sin ella (sección 3.6).

El *speedup* indica cuántas veces más rápido es el programa con P procesos que con uno solo ($S = T_1/T_P$). La *eficiencia* es el speedup dividido por P ($E = S/P$) y vale 1 si el aprovechamiento es perfecto. Se llama *overhead* a la diferencia entre el tiempo medido y el ideal, T_1/P. Según la Ley de Amdahl, la parte del programa que no se puede paralelizar limita el speedup máximo, y el tiempo total se compone de cálculo, comunicación y tiempo ocioso (por esperas o desbalance de carga). Todos los tiempos que se informan son la mediana de las repeticiones de cada combinación, para reducir el efecto de picos ocasionales del sistema.

2.3 Cómo validamos los resultados

En cada corrida se calcula la probabilidad obtenida (victorias de “cambiar” sobre el total de partidas) y se la compara con la teórica, $(M - 1)/M$. Se verifica que la diferencia se achique al aumentar N y que no quede siempre del mismo lado.

3. Resultados

Esta sección presenta lo medido con la versión final del simulador. La única excepción es el problema del generador de números aleatorios (2.1), que apareció en la primera versión.

3.1 ¿Coincide la simulación con la teoría?

Sí. Con mil millones de partidas, cambiar ganó en 0,666666 de las partidas en el caso clásico, frente al valor teórico 0,666667. Con 5, 10 y 50 puertas la coincidencia es igual de buena (Tabla 2), y en el clúster el resultado es el mismo. La diferencia con la teoría se achica al aumentar N, como se espera de un método de Monte Carlo: cada vez que N se multiplica por 100, la diferencia se divide por 10 (Figura 1). Además, no queda siempre del mismo lado: a veces la simulación está apenas por encima de la teoría y a veces apenas por debajo.

Figura 1. Diferencia entre la simulación y la teoría al aumentar N, para 3, 5, 10 y 50 puertas. La línea roja marca el comportamiento esperado.

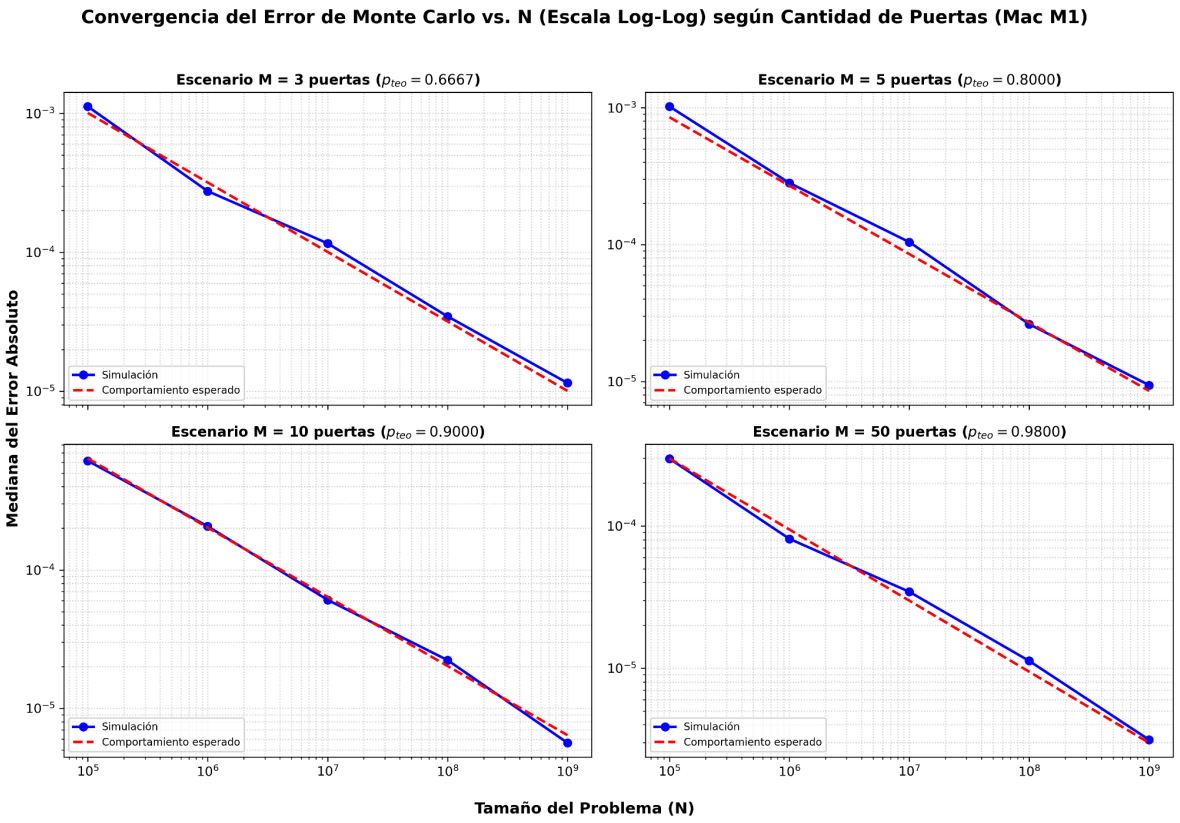
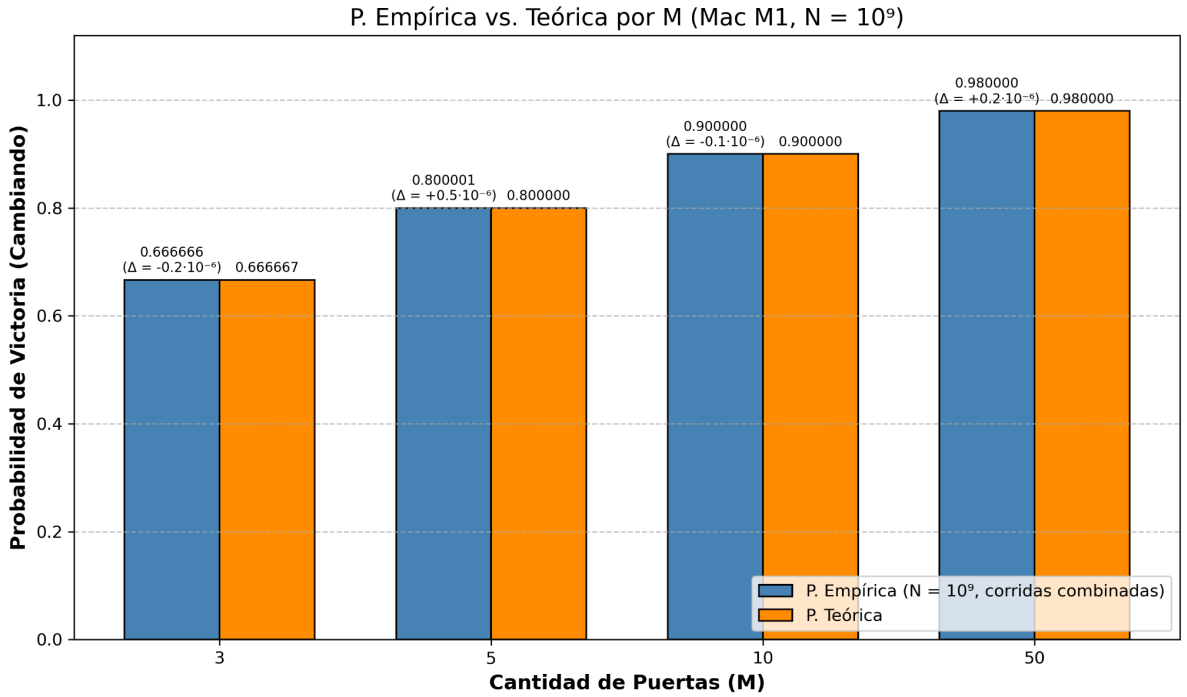


Figura. Probabilidad de ganar cambiando con N = mil millones (Mac, promedio de las 80 corridas de cada M).



3.2 Un problema encontrado: el generador de números aleatorios

La primera versión del simulador usaba la función rand() de C. Con pocas partidas los resultados eran correctos, pero con cientos de millones el valor para 3 puertas quedaba siempre un poco por debajo de 2/3, y esa diferencia dejaba de achicarse al aumentar N. Era de unas cuatro cienmilésimas (0,00004), quedaba del mismo lado en todas las corridas con mil millones de partidas y no dependía de la cantidad de procesos. Con pocas partidas es demasiado chica para notarse entre las variaciones normales de cada corrida. Con mil millones ya las supera.

La causa está en cómo rand() de macOS genera los números: cada uno se calcula a partir del anterior con una fórmula fija ($x' = 16807 \cdot x \bmod (2^{31} - 1)$), por lo que dos números consecutivos no son del todo independientes. El programa sortea con dos números consecutivos la puerta del premio y la puerta elegida, y con 3 puertas esas dos coinciden algo más seguido de lo debido. Con esa fórmula se puede calcular la diferencia de antemano: unas 0,00004, que es lo que se midió (Tabla 3). Al reemplazar rand() por SplitMix64 la diferencia desapareció.

Tabla 3. Diferencia con 2/3 con N = mil millones y 3 puertas.

Fuente	**Diferencia con 2/3**
Cálculo con la fórmula de rand()	−0,0000397
rand() (Mac, 80 corridas)	−0,0000406 (negativa en todas)

SplitMix64 (Mac, 80 corridas)	−0,0000002

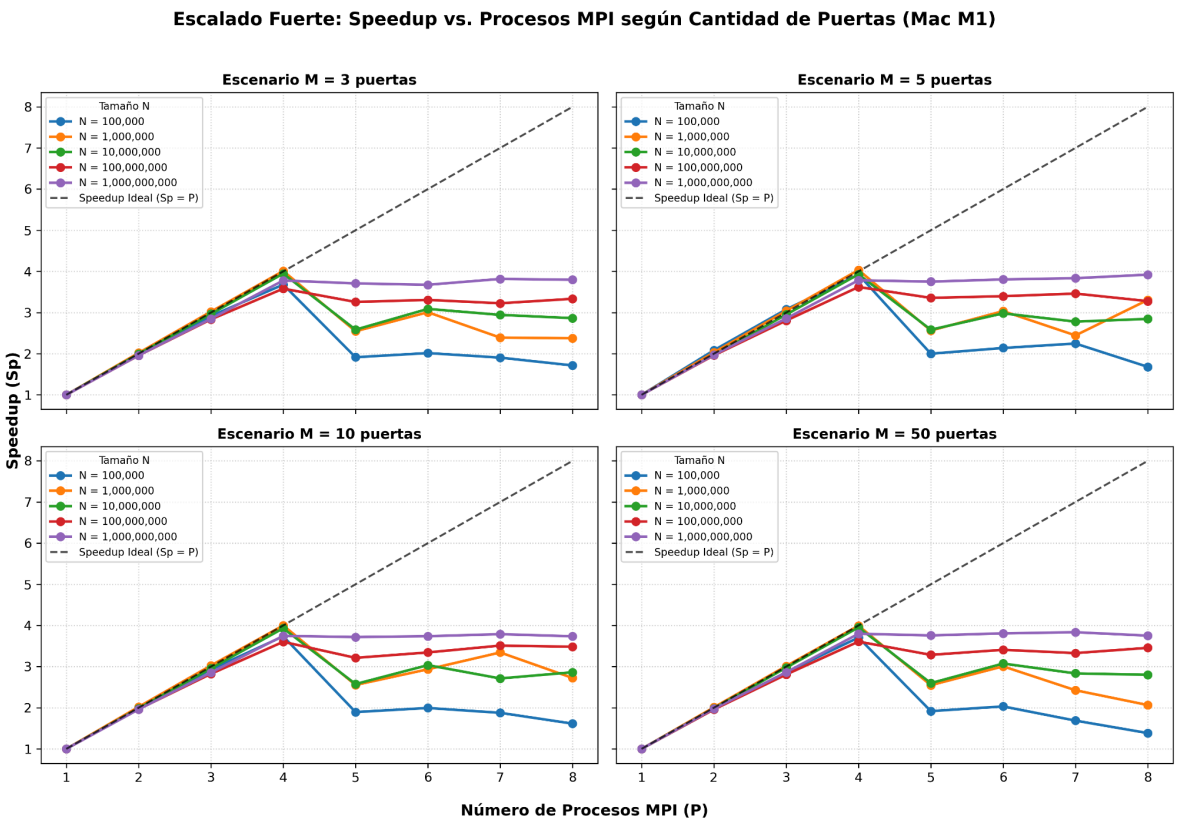
Con 5, 10 y 50 puertas la diferencia calculada es mucho menor (menos de 0,00001), por lo que el caso clásico era el más afectado. Este problema corresponde a rand() de macOS: no se midió en el clúster, donde solo se usó SplitMix64.

3.3 ¿Cómo escala en la Mac?

Hasta 4 procesos el speedup es casi perfecto (3,78 con 4 procesos y mil millones de partidas). Desde 5 se estanca: con 8 procesos es 3,80, igual que con 4 (Figura 2 y Tabla 4). Ocurre igual con 3, 5, 10 y 50 puertas. Para esta máquina, el mejor valor es P = 4.

Una explicación es que la M1 tiene 4 núcleos rápidos y 4 más lentos. Como el trabajo se reparte en partes iguales, los procesos que caen en núcleos lentos terminan más tarde y los demás esperan. Es coherente con que el tiempo perdido con 8 procesos crezca con el tamaño del problema: pasa de 0,031 s con cien millones de partidas a 0,248 s con mil millones, unas 8 veces más, algo que no pasaría si fuera un costo de comunicación.

Figura 2. Speedup en la Mac M1 para 3, 5, 10 y 50 puertas: casi ideal hasta 4 procesos y estancado desde 5.



3.4 ¿Cómo escala en el clúster?

En el clúster el speedup es casi lineal: con 28 procesos y mil millones de partidas el programa fue 27,9 veces más rápido, con una eficiencia de 0,996 (Figura 3 y Tabla 4). Con

problemas chicos la ganancia se frena antes. Con diez mil partidas el speedup deja de crecer cerca de los 15 procesos (llega a unas 8 veces) y después baja. Con cien mil sigue subiendo hasta cerca de 20. Con poco trabajo, el tiempo extra de coordinar a los procesos se vuelve comparable con el de cálculo: entre diez mil y un millón de partidas es de unas decenas de microsegundos (43 a 57 µs con 28 procesos), mientras que con mil millones es el 0,4 % del tiempo total.

Figura 3. Speedup en el clúster con 1 a 28 procesos: casi lineal con problemas grandes.

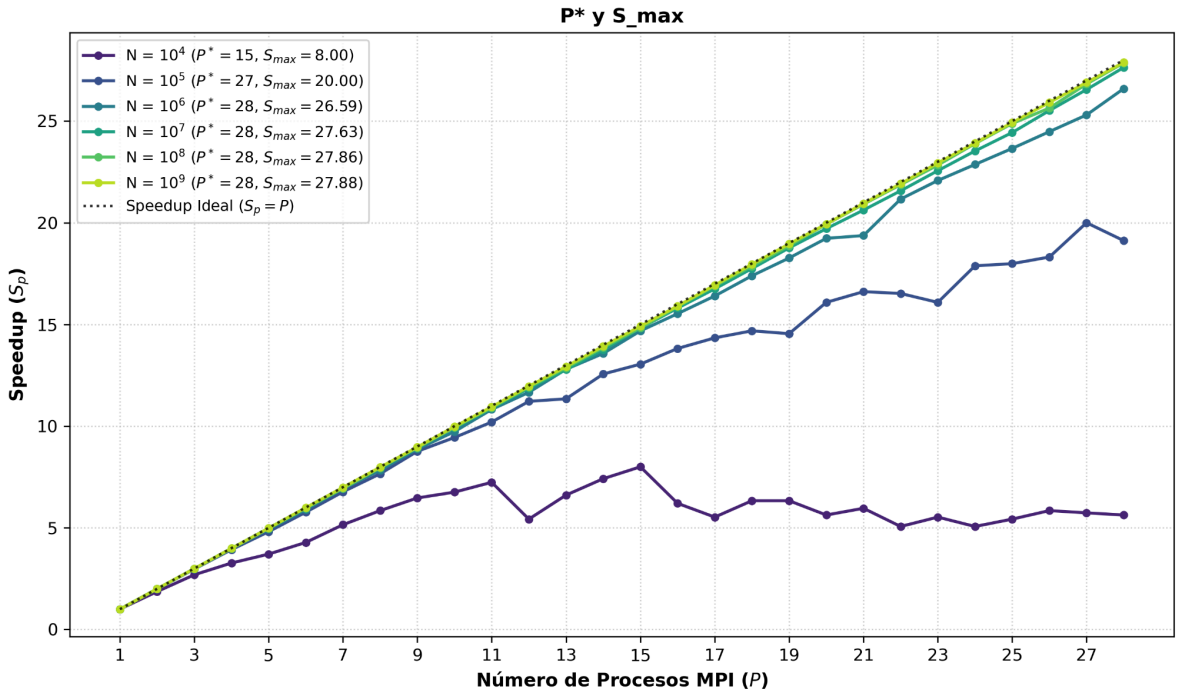


Tabla 4. Rendimiento con N = mil millones y 3 puertas.

Sistema	Procesos	Speedup	Eficiencia
Mac M1	4	3,78	0,945
Mac M1	8	3,80	0,475
Clúster	8	7,98	0,998
Clúster	28	27,88	0,996

Los valores del clúster con 8 procesos vienen del experimento espejo y los de 28 procesos del barrido continuo.

3.5 ¿Cuántos procesos conviene usar según el tamaño del problema?

Depende del tamaño del problema (N) y del sistema (Tabla 5). En el clúster, con diez mil partidas conviene usar cerca de 15 procesos: con más, el speedup baja porque coordinar a los procesos cuesta más de lo que se gana al repartir el cálculo. Con cien mil partidas el speedup sigue subiendo hasta casi los 28 procesos, pero con una eficiencia de 0,68: cada núcleo agregado se aprovecha poco. Desde un millón de partidas el programa escala hasta los 28 procesos sin estancarse. En la Mac conviene usar 4 procesos para cualquier N: con problemas chicos usar 8 empeora el tiempo (speedup de 1,7 con cien mil partidas, frente a 3,7 con 4 procesos) y con problemas muy grandes no mejora ni empeora.

Tabla 5. Cantidad de procesos según N (3 puertas; entre paréntesis, el speedup).

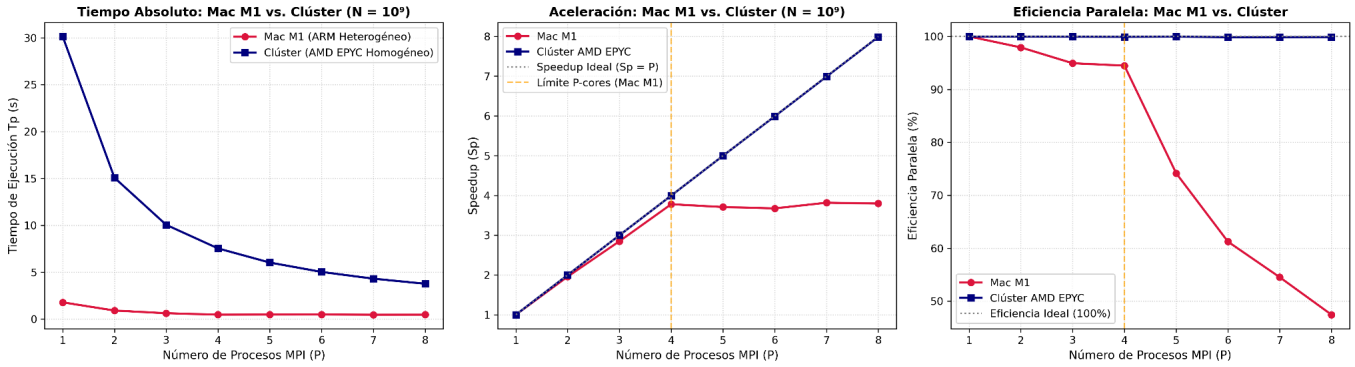
N	Clúster: mejor P	Clúster: eficiencia con 28	Mac: speedup con 4	Mac: speedup con 8
10^4	≈ 15 (8,0)	0,20	no medido	no medido
10^5	27 (20,0)	0,68	3,68	1,7
10^6	28 (26,6)	0,95	4,01	2,4
10^7	28 (27,6)	0,987	3,95	2,9
10^8	28 (27,9)	0,995	3,58	3,3
10^9	28 (27,9)	0,996	3,78	3,80

28 es el máximo del barrido.

3.6 Comparación entre sistemas y otros efectos

Entre sistemas. Comparamos cómo escala cada sistema (Figura 4): con 8 procesos, la eficiencia es 0,475 en la Mac y 0,998 en el clúster.

Figura 4. Tiempo, speedup y eficiencia en la Mac y en el clúster (N = mil millones).



***Cantidad de puertas**:* No cambia el tiempo de ejecución: con mil millones de partidas y un proceso, la diferencia entre 3, 5, 10 y 50 puertas es menor al 0,3 % (Mac).

Versión serial: La versión serial (sin MPI) y la versión MPI con un solo proceso tardan prácticamente lo mismo. Con mil millones de partidas, la serial tardó 1,77 s contra 1,79 s de la versión MPI en la Mac, y 30,4 s contra 30,2 s en el clúster. Con todos los tamaños de problema la diferencia es menor al 2,5 %, salvo en la Mac con cien mil partidas (5,9 %, unos 10 microsegundos). Es decir, usar MPI con un solo proceso no agrega un costo medible, y por eso el tiempo con un proceso sirve como referencia para calcular el speedup.

Barrera previa al cronómetro: Para comprobar si era necesaria, se repitió parte de los experimentos sin ella (3 puertas, 5 repeticiones). Sin barrera, el tiempo medido incluye el desfase con que arrancan los procesos. Con problemas grandes no hay diferencia: con mil millones de partidas, el speedup en la Mac cambia 3,4 % como máximo, a veces para arriba y a veces para abajo (Tabla 6). Con problemas chicos el speedup sin barrera es menor. En la Mac, con cien mil partidas y 8 procesos, cae de 1,71 a 0,54, es decir, más lento que usar un solo proceso. Con un millón de partidas la caída es de 3 % a 9 %. En el clúster, con cien mil partidas y 8 procesos, baja de 7,70 a 5,77, y con un millón apenas de 7,86 a 7,68. Con diez millones de partidas en la Mac los resultados son irregulares, sin un patrón claro. En el clúster solo se probó con cien mil y un millón de partidas y hasta 8 procesos. Por eso se conservó la barrera: mide el costo de la parte paralela sin mezclarlo con el arranque de los procesos.

Tabla 6. Speedup con y sin barrera (3 puertas).

Sistema	N	Procesos	Con barrera	Sin barrera
Mac M1	10^5	4	3,68	2,78
Mac M1	10^5	8	1,71	0,54
Mac M1	10^6	8	2,38	2,30

Mac M1	10^9	8	3,80	3,75
Clúster	10^5	8	7,70	5,77
Clúster	10^6	8	7,86	7,68

3.7 Un tercer sistema: una PC con Windows

Para completar la comparación, se repitió el experimento principal en una computadora con Windows y un procesador AMD Ryzen 5 2600 (6 núcleos físicos y 12 hilos), con N de 10^5 a 10^9, P de 1 a 8, las cuatro cantidades de puertas y 5 repeticiones por combinación. El programa es el mismo que en los otros sistemas, salvo la forma de obtener la semilla inicial: la función clock_gettime no existe en Windows, y se usó en su lugar el reloj de MPI. Con mil millones de partidas, los resultados coinciden con la teoría igual que en los otros sistemas: cambiar ganó en 0,666666 de las partidas con 3 puertas, y con 5, 10 y 50 puertas la diferencia con la teoría es menor a tres millonésimas.

En rendimiento, el escalado es intermedio entre el de la Mac y el del clúster. Con problemas grandes (diez millones de partidas o más) el speedup sigue creciendo hasta 8 procesos, sin el estancamiento de la Mac, pero la eficiencia queda lejos de la del clúster: con mil millones de partidas, el programa fue 5,5 veces más rápido con 8 procesos (eficiencia 0,69; Figura 5 y Tabla 7). Con cien mil partidas el escalado es pobre, como en los otros sistemas con problemas chicos (2,5 veces con 8 procesos).

El procesador tiene 6 núcleos físicos, por lo que con 7 y 8 procesos hay más procesos que núcleos, aunque la PC puede ejecutar 12 hilos a la vez. Los datos lo muestran: con diez millones de partidas o más, el programa es más rápido con 8 procesos que con 6 (5,5 contra 4,4 veces con mil millones de partidas). Cuando dos procesos comparten un mismo núcleo, cada uno anda más lento que si estuviera solo, y por eso el speedup crece cada vez menos. Pero la eficiencia ya baja con 2 a 4 procesos, cuando sobran núcleos, y con 2 procesos es menor cuanto más grande es el problema (0,99 con 10^5 y 0,85 con 10^9), lo contrario de lo que pasó en la Mac y el clúster. La pérdida es proporcional al trabajo y no a la cantidad de procesos: con mil millones de partidas, cada partida tarda unos 40 ns con un solo proceso y unos 59 ns con ocho, y casi todo el aumento ocurre entre 1 y 4 procesos. Por eso no se debe a un costo de coordinación, que sería el mismo para cualquier tamaño de problema.

Figura 5. Speedup y eficiencia en la PC con Windows (3 puertas).

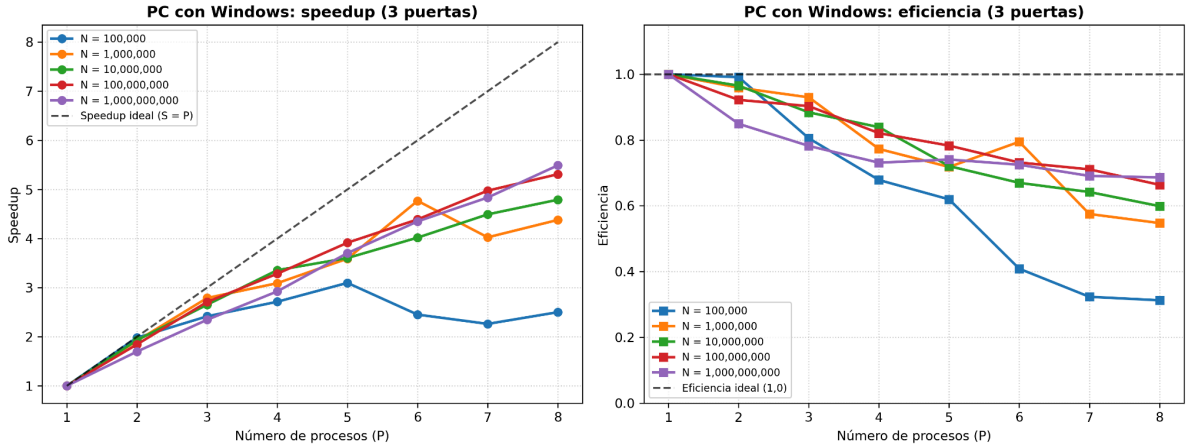


Tabla 7. Speedup y eficiencia en la PC con Windows (3 puertas).

N	Speedup con 4	Speedup con 8	Eficiencia con 4	Eficiencia con 8
10^5	2,71	2,50	0,68	0,31
10^6	3,09	4,38	0,77	0,55
10^7	3,36	4,79	0,84	0,60
10^8	3,28	5,31	0,82	0,66
10^9	2,92	5,49	0,73	0,69

4. Conclusiones

La simulación confirma la teoría: Con mil millones de partidas, cambiar de puerta ganó en 0,666666 de los casos en el juego clásico (la teoría dice 0,666667), y con 5, 10 y 50 puertas los resultados coinciden con 0,800, 0,900 y 0,980. Cambiar de puerta conviene siempre, y conviene más cuanto mayor es la cantidad de puertas. Esto vale cuando el presentador conoce el premio y abre todas las puertas vacías menos una. El programa no reproduce al presentador: usa la equivalencia de que cambiar gana exactamente cuando la elección inicial fue errónea.

La calidad de los números aleatorios importa: La función rand() de macOS producía una diferencia constante de cuatro cienmilésimas con la teoría, que solo se hizo visible con

cientos de millones de partidas. Al reemplazarla por SplitMix64 la diferencia desapareció. Un resultado que parece correcto con pocas partidas puede esconder un error que aparece al aumentar el tamaño del problema.

El problema se paraleliza muy bien, pero cuánto se acelera depende del hardware: En el clúster, con núcleos iguales, el programa fue casi 28 veces más rápido con 28 procesos. En la Mac fue casi ideal hasta 4 procesos y desde 5 se estancó en unas 3,8 veces: con 8 procesos no fue más rápido que con 4. Creemos que se debe a que la Mac mezcla núcleos rápidos y lentos y el trabajo se reparte en partes iguales, pero es una hipótesis que no comprobamos midiendo cada proceso.

El tamaño del problema decide cuántos procesos conviene usar: En el clúster, con diez mil partidas el speedup deja de crecer cerca de los 15 procesos y después baja. Con cien mil sigue creciendo, pero con una eficiencia de 0,68 con 28 procesos. Desde un millón de partidas escala sin estancarse hasta los 28. Con poco trabajo, el tiempo extra de coordinar a los procesos se vuelve comparable con el de cálculo. En la Mac conviene usar 4 procesos para cualquier tamaño.

Otros efectos medidos: La cantidad de puertas no cambia el tiempo de ejecución (menos de 0,3 %). Usar MPI con un solo proceso tarda lo mismo que la versión serial, así que no agrega un costo medible. La barrera previa al cronómetro es necesaria con problemas chicos: sin ella, el tiempo medido incluye el desfase con que arrancan los procesos y el speedup resulta menor, aunque con problemas grandes no hace diferencia.

En términos prácticos: usar 4 procesos en la Mac y todos los núcleos disponibles en el clúster, con un N lo bastante grande para la precisión que se necesite.

4.1 Limitaciones

- Los datos de Windows se tomaron en una PC personal con condiciones no controladas. El clúster fue un único nodo, por lo que no se midió la comunicación entre computadoras distintas.
- No se midió el tiempo de cada proceso, por lo que la explicación de lo que pasa en la Mac es una hipótesis.
- En el barrido del clúster hay solo 3 repeticiones por punto y el reloj mide microsegundos: los resultados con diez mil y cien mil partidas son orientativos.
- No se registró la carga ni la cuota de procesadores del clúster durante las corridas (es un equipo compartido).
- La diferencia de rand() se detectó en la Mac. No se midió en otros sistemas.
- La prueba sin barrera en el clúster abarcó solo cien mil y un millón de partidas y hasta 8 procesos.
- Se estudió únicamente la variante en la que el presentador abre todas las puertas vacías menos una.
- En una PC con Windows el escalado fue intermedio entre la Mac y el clúster, con condiciones menos controladas.

4.2 Mejoras recomendadas

1. **Medir el tiempo de cada proceso**, para separar cálculo, comunicación y espera. Eso confirmaría o descartaría la explicación del estancamiento de la Mac.
2. **Repartir el trabajo según la velocidad de cada núcleo** (por ejemplo, en bloques que cada proceso va pidiendo) para aprovechar los núcleos lentos de la Mac.
3. **Probar en varios nodos del clúster** para medir el costo de la red, con problemas más grandes, más repeticiones en los tamaños chicos y registrando la carga del equipo.
4. **Usar un generador pensado para programas paralelos**, como xoshiro256** (con saltos) o Random123, que generan secuencias que no se solapan, y repetir la prueba de rand() en otros sistemas para saber si la diferencia es propia de macOS.
5. **Medir también el tiempo total del programa**, que incluye el arranque de los procesos, para cuantificar su costo (con la barrera medimos solo la parte paralela).
6. **Estudiar la variante en la que el presentador abre una sola puerta.**

5. Referencias

- Teoría proporcionada por la cátedra sobre HPC.
- Blackman, D., & Vigna, S. (2021). Scrambled linear pseudorandom number generators. *ACM Transactions on Mathematical Software, 47*(4), 36:1–36:32.
- Message Passing Interface Forum. (2015). *MPI: A message-passing interface standard, version 3.1*.
- Salmon, J. K., Moraes, M. A., Dror, R. O., & Shaw, D. E. (2011). Parallel random numbers: As easy as 1, 2, 3. En *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*. ACM.
- Steele, G. L., Jr., Lea, D., & Flood, C. H. (2014). Fast splittable pseudorandom number generators. En *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14)*(pp. 453–472). ACM.
- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *AFIPS Conference Proceedings* (Vol. 30, pp. 483–485). AFIPS Press.
- Park, S. K., & Miller, K. W. (1988). Random number generators: Good ones are hard to find. *Communications of the ACM, 31*(10), 1192–1201.